	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: R.Sai Manasa Reg. No:2503A52462
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 06-10-2026

Task 1 : improve readability

1. Original code :

```
a = 80
b = 70
c = 90
x = (a + b + c) / 3
if x >= 50:
print("pass")
else: print("fail")
```

2. Prompt : Refactor this Python code to improve readability. Use meaningful variable names, create a function called calculate_average(), and improve the formatting. Make sure the refactored code produces the same output as the original code.

3. Ai suggestions: Replace a, b, and c with meaningful names such as mark1, mark2, and mark3. Replace x with average. Create a reusable calculate_average() function. Use proper indentation and spacing. Keep the pass/fail condition unchanged so the behavior remains the same.

4. Refactered code :

```
def calculate_average(score_one, score_two, score_three):
  return (score_one + score_two + score_three) / 3
average_score = calculate_average(80, 70, 90)
if average_score >= 50:
   print("pass")
else:
   print("fail")
```

5. Test output :

Pass

6. Explanation :

Meaningful variable names:

a, b, c, and x were changed to mark1, mark2, mark3, and average.

Reusable function:

The calculation was moved into calculate_average(), making the code reusable.

Better formatting:

Proper indentation and blank lines make the program easier to read.

Same behavior:

The refactored program still calculates the average and prints pass when the average is at least 50.

Easier maintenance:

If the average calculation needs to be changed later, it can be modified inside the function instead of throughout the program.

Task 2 : – Remove Repeated Code

1. **Original code :**

```
name1 = "Arun"
marks1 = 85
```

```
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

prompt :

Identify the duplicated logic in this Python code and refactor it into a reusable function. The function should accept a student's name and marks and print whether the student passed or failed. Test the function with at least four students and make sure the output remains correct.

Ai suggestions:

Suggested improvement to Create a reusable function such as:

```
def check_result(name, marks):
```

Then call the function for each student.

Refracted code :

```
def check_result(name, marks):
  if marks >= 50:
    print(name, "Pass")
  else:
    print(name, "Fail")
check_result("Arun", 85)
check_result("Priya", 72)
check_result("Rahul", 45)
check_result("Meena", 60)
```

test output :

Arun Pass

Priya Pass

Rahul Fail

Meena Pass

Explanation : .

Less code:

The pass/fail logic is written only once. .

Easy to modify:

If the passing mark changes from 50 to 40, we only change it inside the function. .

Reusable:

The same function can be used for any number of students.

Fewer errors:

We don't need to repeatedly write the same if-else logic.

Easier to understand:

The function clearly represents the task of checking a student's result.

Task 3 : Refactor a Long Conditional

Original code :

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

prompt :

Simplify the conditional logic in this Python program. Create a function called determine_priority(age, temperature) that produces exactly the same results as the original code. Test it with at least four different age and temperature combinations.

Ai suggestions :

The original code has repeated conditions.

The important rules are:

- Age ≥ 60 AND temperature ≥ 38 → High Priority
- Age ≥ 60 OR temperature ≥ 38 → Medium Priority
- Otherwise → Normal Priority

So the conditional can be simplified.

Refactored code :

```
def determine_priority(age, temperature):
 if age >= 60 and temperature >= 38:
    return "High Priority"
 elif age >= 60 or temperature >= 38:
    return "Medium Priority"
 else:
    return "Normal Priority"
age = 68
temperature = 39
print(determine_priority(age, temperature))
```

test output :

High Priority.

Explanation :

The AI preserved the original behavior.

The refactored code gives:

- Older person + high temperature → High Priority
- Older person + normal temperature → Medium Priority
- Younger person + high temperature → Medium Priority
- Younger person + normal temperature → Normal Priority

Therefore, the refactoring simplifies the code without changing its output.

TASK 4 – Refactor Inefficient List Processing

Original code :

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

prompt :

Improve the readability and efficiency of this Python code. Determine whether a list, set, or dictionary is better for frequent patient ID lookups. Refactor the code using the most suitable data structure. Test it with both an existing and a missing patient ID.

AI suggestions : The original code searches through the entire list using a for loop. A set is better when we frequently check whether an ID exists because membership checking with a set is typically O(1) on average.

Refracted code :

```
patient_ids = {101, 205, 309, 412, 518, 623}
target = 412
if target in patient_ids:
    print("Patient Found")
else:
    print("Patient Not Found")
```

test output :

Patient Found

Explanation :

The original code uses a for loop and a found variable to search for the patient ID.

The refactored version:

- Removes the unnecessary loop.
- Removes the found variable.
- Uses a set for faster lookups.
- Makes the code shorter and easier to understand.
- Correctly handles both existing and missing patient IDs.

TASK 5 – Complete Legacy Code Review with AI

Original code :

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

prompt :

Review the following legacy Python code and identify at least five improvements. Refactor the code using meaningful variable names, reusable functions, and suitable Python features where they improve clarity. Make sure the refactored code produces the same output as the original code. Also explain which improvements you accepted or rejected and why.

AI Suggestions:

1.Use meaningful variable names

Replace unclear names like s, avg, and total with descriptive names such as student, average_marks, and total_marks.

2.Create a calculate_average() function

Move the average calculation into a reusable function so it can be easily called whenever needed.

3.Create a display_results() function

Move the pass/fail checking logic into a separate function to make the program more organized and readable.

4.Use Python's sum() function

Instead of manually adding marks using a loop, use sum() to calculate the total marks more clearly and concisely.

5. Use tuple unpacking

Instead of accessing student data using s[0] and s[1], use name, marks = student to make the code easier to understand.

Refractored :

```
students = [
("Arun", 78),
("Priya", 45),
("Rahul", 88),
("Meena", 52)
]
def calculate_average(students):
  total_marks = sum(marks for name, marks in students)
  return total_marks / len(students)
def display_results(students):
  for name, marks in students:
    if marks >= 50:
      print(name, "Pass")
    else:
      print(name, "Fail")
display_results(students)
average = calculate_average(students)
print("Average =", average)
```

test output :

Arun Pass

Priya Fail

Rahul Pass

Meena Pass

Average = 65.75

Explanation

The refactored code improves the original program in several ways:

1. **Meaningful variable names**
   Names like student, marks, total_marks, and average clearly describe what each value represents. This makes the code easier to understand.
2. **Reusable functions**
   calculate_average() handles the average calculation, while display_results() handles the pass/fail results. Dividing the program into functions makes it easier to maintain and reuse.

3. **Use of sum()**

   Instead of manually adding every student's marks using a loop, sum() calculates the total directly. This makes the code shorter and clearer.

4. **Tuple unpacking**

   Using name, marks instead of s[0] and s[1] makes it immediately clear which value represents the student's name and which represents the marks.